Hadoop & Spark - ISEN 2025/2026

Lab: Work with multiple Dataframes

For this Lab you will need:

- The 3.9 Lab

As a reminder

Check 3.1 to login in NGROK, Kaggle, install Java or Spark and import libraries.

Preparation

As usual, let's load the Taxi dataset if not already done:

```
!kaggle datasets download -d microize/newyork-yellow-taxi-trip-data-2020-2019
!unzip newyork-yellow-taxi-trip-data-2020-2019.zip
```

Preparation

```
yellowTaxiSchema = (
                        StructType
                        ([
                            StructField("VendorId"               , IntegerType()   , True),
                            StructField("PickupTime"             , TimestampType() , True),
                            StructField("DropTime"               , TimestampType() , True),
                            StructField("PassengerCount"         , DoubleType()    , True),
                            StructField("TripDistance"           , DoubleType()    , True),
                            StructField("RateCodeId"             , DoubleType()    , True),
                            StructField("StoreAndFwdFlag"        , StringType()    , True),
                            StructField("PickupLocationId"       , IntegerType()   , True),
                            StructField("DropLocationId"         , IntegerType()   , True),
                            StructField("PaymentType"            , IntegerType()   , True),
                            StructField("FareAmount"             , DoubleType()    , True),
                            StructField("Extra"                  , DoubleType()    , True),
                            StructField("MtaTax"                 , DoubleType()    , True),
                            StructField("TipAmount"              , DoubleType()    , True),
                            StructField("TollsAmount"            , DoubleType()    , True),
                            StructField("ImprovementSurcharge"   , DoubleType()    , True),
                            StructField("TotalAmount"            , DoubleType()    , True),
                            StructField("CongestionSurcharge"    , DoubleType()    , True),
                        ])
                    )
```

Preparation

Read and create temp view:

```
yellowTaxiDF = (
                  spark
                    .read
                    .option("header", "true")
                    .schema(yellowTaxiSchema)
                    .csv("yellow_tripdata_2020-04.csv")
                )
yellowTaxiDF.createOrReplaceTempView("YellowTaxis")
yellowTaxiDF.printSchema()
```

Preparation

Then load the taxi zones:

```
taxiZonesSchema = "PickupLocationId INT, Borough STRING, Zone STRING, ServiceZone STRING"
taxiZonesDF = (
                  spark
                    .read
                    .schema(taxiZonesSchema)
                    .csv("taxi+_zone_lookup.csv")
               )
taxiZonesDF.createOrReplaceTempView("TaxiZones")
taxiZonesDF.printSchema()
```

Joins

Display location information for each ride:

> Note: if there is more than one condition, you can specify it with an array [condition1, condition2...]

```
joinedDF = (
                yellowTaxiDF
                    .join
                    (
                        taxiZonesDF,
                        yellowTaxiDF.PickupLocationId == taxiZonesDF.PickupLocationId, # [condition1, condition2...]
                        "inner"   # left, leftouter, right, rightouter, full etc.
                    )
            )

joinedDF.printSchema()
```

Joins

Please notice that `PickupLocationId` appears twice.

You may select your columns with alias to prevent it:

```
joinedDF = (
                yellowTaxiDF.alias("yt")
                    .join
                    (
                        taxiZonesDF.alias("tz"),
                        col("yt.PickupLocationId") == col("tz.PickupLocationId"),
                        "inner"
                    ).drop(col("tz.PickupLocationId"))
            )

joinedDF.printSchema()
```

```
joinedDF.show(5)
```

Exercise Join

Find all LocationIds in TaxiZones, from where no pickups have happened (ordered by PickupLocationId)

> **Hint:** Think about what join operation you could do between `TaxiZones` & `YellowTaxis`

Sets

For this we will need to upload the `Driver.csv` file from the data folder of this course.

```
from google.colab import files
files.upload()
```

Sets

Read it and create a view

```
driversDF = (
                  spark
                    .read
                    .option("header", "true")
                    .option("inferSchema", "true")
                    .csv("Drivers.csv")
             )
driversDF.createOrReplaceTempView("Drivers")

driversDF.show()
```

Sets

We will load the Cab file aswell once again:

```
cabsDF = (
              spark
                .read
                .option("header", "true")
                .option("inferSchema", "true")
                .csv("Cabs.csv")
         )

cabsDF.createOrReplaceTempView("Cabs")

cabsDF.show()
```

> You can see the unique name of drivers in the `Name` column of both datasets

Sets

Let's create a list of all drivers in SQL using union all

```
all_drivers = (spark.sql("""

    (
        SELECT DISTINCT Name
        FROM Cabs
    )

    UNION ALL

    (
        SELECT DISTINCT Name
        FROM Drivers
    )

"""))
print(all_drivers.count())
all_drivers.show()
```

Sets

Let's create a list of all unique drivers in SQL using union

```
distinct_drivers = (spark.sql("""

    (
        SELECT DISTINCT Name
        FROM Cabs
    )

    UNION

    (
        SELECT DISTINCT Name
        FROM Drivers
    )

"""))
print(distinct_drivers.count())
distinct_drivers.show()
```

Sets

Let's create a list of all registered drivers who are driving cab

```
registered = (spark.sql("""

    (
        SELECT DISTINCT Name
        FROM Cabs
    )

    INTERSECT

    (
        SELECT DISTINCT Name
        FROM Drivers
    )

"""))
print(registered.count())
registered.show()
```

Sets

Let's create a list of all unregistered (meaning they are in the list of cabs drivers but not in our driver list) drivers who are driving cab

```
unregistered = (spark.sql("""

    (
        SELECT Name
        FROM Cabs
        WHERE LicenseType = 'OWNER MUST DRIVE'
    )

    EXCEPT

    (
        SELECT Name
        FROM Drivers
    )

"""))
print(unregistered.count())
unregistered.show()
```

Exercise: Sets Operations

Using the Yellow Taxi dataset and Taxi Zones dataset, find the list of PickupLocationId that are in one dataset an not the other.

> **Hint:** Think about what set operation you could do between `TaxiZones & YellowTaxis`